

unplugin-auto-import插件项目使用指南

unplugin-auto-import的核心作用是「自动导入API」（如Vue的ref、onMounted，uni-app的uni.showToast等），无需手动写import语句，大幅提升开发效率（improve development efficiency significantly）。以下分「通用Vue项目」和「uni-app项目」两种场景，详细说明完整使用流程（适配Vite构建工具，你当前用的是Vite，重点看对应场景）。

一、前置准备：安装插件

无论哪种场景，第一步均需安装插件及依赖，打开项目根目录的终端，执行以下命令：

Code block

```
1  npm install unplugin-auto-import -D
2  # 或 yarn/pnpm (根据你的包管理器选择)
3  yarn add unplugin-auto-import -D
4  pnpm add unplugin-auto-import -D
```



说明：-D 表示安装为开发依赖（仅开发时用，打包后不包含），符合插件的工具属性。

二、核心配置：适配不同项目场景

配置文件为项目根目录的 `vite.config.js`（或 `vite.config.ts`），核心差异在于「项目核心插件」和「自动导入的API范围」，需根据项目类型选择对应配置。

场景1：uni-app项目（你的当前场景）

uni-app需使用专属的Vite插件 `@dcloudio/vite-plugin-uni`，自动导入范围需包含「Vue基础API」和「uni-app专属API」，配置如下（可直接复制替换原有配置）：

Code block

```
1  import { defineConfig } from 'vite'
2  // 引入uni-app的Vite核心插件 (必须)
3  import uni from '@dcloudio/vite-plugin-uni'
4  // 引入unplugin-auto-import
5  import AutoImport from 'unplugin-auto-import/vite'
6
7  export default defineConfig({
8    plugins: [
9      // 1. 先配置uni插件 (uni-app项目的核心, 必须放在前面)
```

```

10 uni(),
11 // 2. 配置auto-import
12 AutoImport({
13   // 自动导入的API范围: Vue基础API + uni-app专属API
14   imports: [
15     'vue', // 包含ref、reactive、onMounted等Vue核心API
16     'uni-app' // 包含uni.showToast、uni.request、uni.navigateTo等uni-app API
17   ],
18   // 关键配置: 生成类型声明文件 (.d.ts), 让编辑器识别自动导入的API, 避免标红
19   dts: true,
20   // 可选: 自定义声明文件路径 (默认在根目录auto-imports.d.ts)
21   // dts: 'src/auto-imports.d.ts',
22   // 可选: 忽略自动导入的API (避免和手动导入冲突, 新手暂时不用)
23   // ignores: ['ref']
24 })
25 ]
26 })

```

场景2: 通用Vue项目 (非跨端, 纯Vue+Vite)

纯Vue项目使用官方Vite插件 `@vitejs/plugin-vue`, 若用到路由 (vue-router), 需同时导入路由API, 配置如下:

Code block

```

1  import { defineConfig } from 'vite'
2  // 引入Vue的Vite核心插件
3  import vue from '@vitejs/plugin-vue'
4  // 引入unplugin-auto-import
5  import AutoImport from 'unplugin-auto-import/vite'
6
7  export default defineConfig({
8    plugins: [
9      // 1. 配置Vue核心插件
10     vue(),
11     // 2. 配置auto-import
12     AutoImport({
13       imports: [
14         'vue', // Vue基础API
15         'vue-router' // 路由API (如useRoute、useRouter, 若项目用了vue-router才需要)
16       ],
17       dts: true, // 开启类型声明, 优化编辑器提示
18     })
19   ]
20 })

```

三、验证配置：确认插件生效

配置完成后，必须「重启Vite开发服务」（修改vite.config后需重启才生效），然后通过以下步骤验证配置是否成功（verify if the configuration is successful）：

步骤1：查看自动生成的声明文件

重启服务后，项目根目录会自动生成 `auto-imports.d.ts` 文件（若自定义了路径则去对应位置找），文件内容类似这样（自动生成，无需修改）：

Code block

```
1 // Generated by 'unplugin-auto-import'
2 export {}
3 declare global {
4   const ref: typeof import('vue')['ref']
5   const reactive: typeof import('vue')['reactive']
6   const onMounted: typeof import('vue')['onMounted']
7   const uni: typeof import('uni-app')['uni'] // uni-app场景才有这行
8 }
```

该文件的作用是告诉编辑器：这些API是全局可用的，无需手动导入。

步骤2：在组件中测试使用

在任意.vue文件的script setup中，直接使用自动导入的API（无需写import），若能正常运行无报错，则说明生效。

示例（uni-app场景）：

Code block

```
1 <template>
2   <view @click="handleClick">测试自动导入</view>
3 </template>
4
5 <script setup>
6 // 无需写: import { ref, onMounted } from 'vue'
7 // 无需写: import { uni } from 'uni-app'
8 const count = ref(0)
9
10 // 直接使用Vue的生命周期API
11 onMounted(() => {
12   console.log('页面挂载完成, count初始值: ', count.value)
13 })
14
```

```
15 // 直接使用uni-app的API
16 const handleClick = () => {
17   count.value++
18   uni.showToast({
19     title: `点击了${count.value}次`,
20     icon: 'success'
21   })
22 }
23 </script>
```

运行项目后，点击视图能正常弹出提示，控制台无“未定义”错误，即为生效。

四、关键注意事项（新手必看）

1. **插件顺序不能乱**：uni() 或 vue() 等项目核心插件，必须放在 AutoImport() 前面，否则会导致编译异常（核心插件需先处理项目基础逻辑）。
2. **必须重启开发服务**：修改vite.config.js后，一定要重启Vite（比如HBuilderX中停止运行再重新运行），否则配置不生效。
3. **dts: true建议必开**：即使项目用的是JavaScript（非TypeScript），开启后编辑器能提供API补全、参数提示，避免“代码标红但能运行”的困惑，大幅提升开发体验。
4. **避免API命名冲突**：若手动定义了和自动导入API同名的变量（如const ref = 123），会导致冲突报错，需修改自定义变量名。
5. **仅支持模块化API**：自动导入仅对ES模块风格的API生效（如Vue3的组合式API、uni-app的uni.xxx），Vue2的选项式API（如data、methods）无需也无法用该插件自动导入。

五、常见问题排查

- 问题1：编辑器标红“ref未定义”？—— 检查是否开启dts: true，是否生成了auto-imports.d.ts，若已生成，可重启编辑器（如HBuilderX）刷新类型识别。
- 问题2：运行项目报错“Cannot find module 'unplugin-auto-import/vite’”？—— 未安装插件或安装失败，重新执行安装命令，检查package.json中是否有unplugin-auto-import依赖。
- 问题3：uni-app项目编译报错？—— 确认vite.config中是uni()而非vue()，且uni插件放在AutoImport前面。

（注：文档部分内容可能由 AI 生成）